

PROJECT REPORT

EMBEDDED SYSTEMS & IoT DEVICE DESIGN

Project Title: Smart IoT Temperature Telemetry & Cloud Logging System

Task Number: Embedded System Task-4

Intern Id: MT8120

Name: Ujjwal Kumar

Role: Junior Embedded Engineer (Intern)

SYSTEM SPECIFICATIONS & STACK

1. SYSTEM SPECIFICATIONS & COMPONENT MAPPING

The execution of this IoT cloud integration task utilizes a specific set of hardware behaviors modeled precisely within the virtual engine environment. Below are the design configurations:

1.1 Core Hardware Stack:

- **Microcontroller Board:** ESP32 DevKit v4 (Leveraged for its native network layer stack to enable TCP/IP cloud data transmission).
- **Climate Telemetry Sensor:** DHT22 Digital Temperature Sensor (High-precision relative ambient sensor mapped to read local thermodynamics).
- **Virtual Network Layer:** Integrated Wi-Fi Modem (Configured to securely handshake with the cloud server gateway via Wokwi-GUEST access point).

1.2 Software & Environment Stack:

- **Development Environment:** Wokwi Online Virtual Simulation Framework.
- **Core Engine Language:** Embedded C++ (Compiled on the native Arduino-ESP32 Core API layer utilizing <WiFi.h> and <DHT.h> libraries).
- **Remote Target Server:** ThingSpeak IoT Cloud Analytics Platform (Operating via structured REST API endpoints).
- **Communication Interface:** Virtual UART via Serial Port (Configured at 115200 Baud Rate for real-time local telemetry logging).

TECHNICAL IMPLEMENTATION & CODE

2. CIRCUIT DIAGRAM

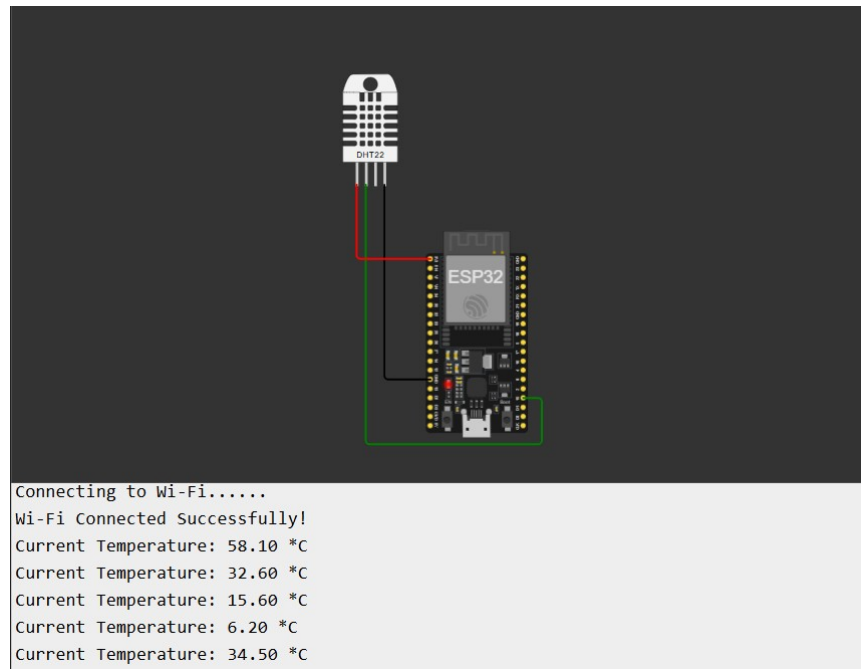


Figure: ESP32 DevKit wired to the DHT22 temperature sensor (VCC to 3V3, DATA to GPIO 15 with pull-up resistor, GND to GND), with the live serial monitor confirming Wi-Fi connection and real-time temperature readings.

2.1 Cloud Channel Configuration (ThingSpeak)

A dedicated private channel named "Smart Weather System" was provisioned on the ThingSpeak IoT Analytics Platform to serve as the remote data-logging endpoint. The channel is configured with a data field to receive the temperature telemetry, and its unique Write API Key is embedded in the firmware to authenticate every upload.

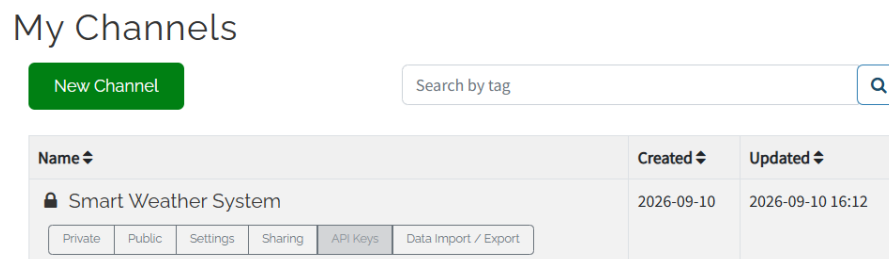


Figure: ThingSpeak "My Channels" dashboard showing the provisioned "Smart Weather System" channel, created on 2026-09-10, used as the cloud destination for real-time sensor telemetry.

3. SOURCE CODE

```
#include <WiFi.h>
```

```

#include <DHT.h>

// Default Wi-Fi credentials for Wokwi virtual simulator environment
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// ThingSpeak target cloud infrastructure server address
const char* server = "api.thingspeak.com";

// Authorized unique Write API Key configured for the cloud channel
String apiKey = "0X46QX4FOXYGYRCM";

#define DHTPIN 15 // Microcontroller digital pin mapping: DHT22
connected to GPIO 15
#define DHTTYPE DHT22 // Initializing the specific physical sensor
type module variant

DHT dht(DHTPIN, DHTTYPE);
WiFiClient client;

void setup() {
    Serial.begin(115200); // Initializing hardware UART Serial
    communication at 115200 Baud Rate
    dht.begin(); // Activating and stabilizing the digital
    temperature sensor hardware

    // Executing dynamic Wi-Fi network layer authentication and connectivity logic
    Serial.print("Connecting to Wi-Fi");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWi-Fi Connected Successfully!"); // Gateway connection established
    successfully
}

void loop() {
    // Reading environmental ambient temperature data streams from the sensor array
    float temp = dht.readTemperature();

    // Data integrity check: Terminate loop routine if data is corrupted or null
    if (isnan(temp)) {
        Serial.println("Failed to read from sensor!");
        return;
    }

    Serial.print("Current Temperature: ");
    Serial.print(temp);
    Serial.println(" *C");

    // Initiating remote client socket connection framework to server on standard HTTP
    Port 80
    if (client.connect(server, 80)) {
        // Constructing the encoded payload string targeting database schema configuration
        Field 1
        String postStr = apiKey + "&field1=" + String(temp) + "\r\n\r\n";

        // Formatting and dispatching the legal standard HTTP POST network requests
        pipeline
        client.print("POST /update HTTP/1.1\n");
    }
}

```

```

    client.print("Host: api.thingspeak.com\n");
    client.print("Connection: close\n");
    client.print("X-THINGSPEAKAPIKEY: " + apiKey + "\n");
    client.print("Content-Type: application/x-www-form-urlencoded\n");
    client.print("Content-Length: " + String(postStr.length()) + "\n\n");
    client.print(postStr);          // Pushing the final telemetry data block
payload onto the cloud

    Serial.println("Data sent to ThingSpeak Server!");
}
client.stop();                    // Gracefully closing the network TCP connection
socket channel

// Enforcing mandatory 15-second delay to comply with ingestion rate limit
constraints
delay(15000);
}

```

RESULTS & WORKING EXPLANATION

4. WORKING EXPLANATION

This project successfully demonstrates how an embedded system can extend a local automation setup into a fully cloud-connected IoT telemetry node, transmitting live sensor readings to a remote dashboard over the internet.

The system continuously reads ambient temperature data using a DHT22 digital temperature sensor. This real-time data stream is processed by the central core of the ESP32 microcontroller, which is also responsible for managing the Wi-Fi network stack.

A sequential telemetry pipeline is executed programmatically:

- **Sensor Acquisition:** The ESP32 polls the DHT22 sensor via the `dht.readTemperature()` function. A data-integrity check using `isnan()` ensures corrupted or null readings are discarded before transmission.
- **Network Connectivity:** On boot, the ESP32 authenticates with the Wokwi-GUEST virtual access point using the `WiFi.begin()` routine, establishing a TCP/IP-capable network layer required for cloud communication.
- **Cloud Transmission:** Once a valid temperature reading is obtained, the microcontroller opens a socket connection to `api.thingspeak.com` on Port 80 and dispatches a structured HTTP POST request containing the sensor value under Field 1, authenticated using the channel's unique Write API Key.
- **Rate-Limit Compliance:** The routine enforces a mandatory 15-second delay between transmissions to comply with ThingSpeak's free-tier ingestion rate limit, after which the loop repeats indefinitely.

The result is a live, self-updating cloud dashboard on ThingSpeak that graphs the temperature trend over time, enabling real-time remote monitoring of the device from any location with internet access.

5. SERIAL MONITOR MONITORING & OUTPUT

```
Connecting to Wi-Fi.....  
Wi-Fi Connected Successfully!  
Current Temperature: 58.10 *C  
Data sent to ThingSpeak Server!  
Current Temperature: 32.60 *C  
Data sent to ThingSpeak Server!  
Current Temperature: 15.60 *C  
Data sent to ThingSpeak Server!
```